A B N H O M E S

ABN Homes Operations Platform

Complete build documentation — what has been built, how it works, and what comes next

Prepared for: Frank Koomson — ABN Homes

Platform: abnhomes.vercel.app (live production)

Date: 11 November 2026

Status: Prototype live · payments and SMS in simulation mode

Table of Contents

1. Executive Summary.....

2. What ABN Homes Is.....1

3. System Architecture.....2

4. What Has Been Built.....3

    4.1 Public Website.....3

    4.2 Marketplace: Listings, Search and Reviews.....3

    4.3 Booking Engine.....4

    4.4 Payments.....5

    4.5 Money: the Double-Entry Ledger.....6

    4.6 Operations: Housekeeping, Maintenance and Assets.....7

    4.7 Guest Experience.....7

    4.8 Messaging and Notifications.....8

    4.9 Admin Console.....8

    4.10 Portals: Owner, Tenant and Vendor.....9

    4.11 Automation and Scheduled Jobs.....10

    4.12 Security and Integrity.....11

5. How It Works: the Guest Journey End to End.....11

6. How It Works: Operations Journeys.....12

7. Data Model Overview.....13

8. Deployment and Operations.....13

9. Verification and Quality.....13

10. What Is Yet to Be Added.....14

    10.1 Go-Live Switches (before real customers).....14

    10.2 Product Roadmap.....15

    10.3 Known Prototype Caveats.....15

Appendix A. Route Inventory (verified 11 November 2026).....15

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

ABN Homes now runs on a single, self-built operations platform that replaces spreadsheets, WhatsApp threads and paper ledgers with one accountable system. The platform is live in production at abnhomes.vercel.app and serves four audiences from one codebase: prospective guests browsing and booking serviced apartments, staff running daily operations from a 22-page admin console, owners tracking performance and payouts through their own portal, and tenants managing leases, trips and requests. Every cedi that moves is recorded in a double-entry ledger that the database itself protects from tampering.

This document records what has been built, explains how each part works, and sets out what remains before and after commercial launch. It is written for the platform owner and any future developer taking over the system. Two companion deliverables accompany it: an illustrated slide deck that walks through the product visually, and a narrated demonstration video that shows every flow end to end on the live production system.

The platform passed a full verification pass on 11 November 2026: all 33 public, admin and portal pages return correct responses; a complete guest journey executed against the production database produced a confirmed booking, two ledger allocations, three guest messages, three staff notifications and a clean refund reversal; and the 37-test suite plus a production end-to-end script are committed to the repository. Payments and SMS run through simulation rails until merchant keys are configured, which keeps the prototype free to operate while leaving nothing structural to change at go-live.

2. What ABN Homes Is

ABN Homes is an Accra-based hospitality and property business that operates two product lines from the same portfolio: short-stay serviced apartments rented by the night, and long-let residential leases rented by the year. The business also performs facility management for the properties it runs — housekeeping, maintenance, security and utility management — and reports to property owners on income and expenses.

Before the platform existed, those functions lived in disconnected tools: bookings arrived by phone and WhatsApp, availability lived in one person's head, rent tracking was a spreadsheet, and owner reporting was a monthly manual exercise. The platform's objective is to make each of those functions self-service where the public is concerned and one-touch where staff are concerned, while keeping a single source of truth for money and dates.

The scope of this build covers the full commercial loop: marketing site, marketplace, booking, payment, confirmation, guest communications, check-in, housekeeping turnover, maintenance, owner reporting and payouts, long-let rent invoicing, and the administrative

console that ties them together. What the build deliberately does not yet include — live payment gateway credentials, an SMS sender account, and a handful of roadmap features — is catalogued in Section 10.

3. System Architecture

The platform is a single Next.js 16 application using the App Router, written in TypeScript, with React Server Components doing the heavy lifting. Next.js server actions handle every write from the browser, which means there is no separate API surface to keep in sync for form flows — the same function that validates a booking is the one the form calls. Prisma 6 is the data layer, speaking to a PostgreSQL 17 database hosted on Supabase through a pooled connection.

Hosting is Vercel for the application and Supabase for the database, with GitHub Actions keeping the database warm on a schedule (a necessary courtesy on Supabase free tier, which pauses idle databases). Authentication is NextAuth v5 with two credential providers: staff sign in with email and password; guests, owners and vendors sign in with a six-digit one-time code sent to a registered phone. Two config files split NextAuth cleanly — a middleware-safe config for route protection and a server config carrying the providers.

Money correctness is enforced below the application layer. The database carries exclusion constraints and trigger guards — defined in prisma/sql/001_constraints.sql and applied after every schema push — that make double bookings physically impossible and make ledger entries append-only. The application can hold 20-second interactive transactions (raised from Prisma's 5-second default after a production incident described in Section 9), and idempotency keys on payment and booking writes make double-submission harmless.

Table 1: Platform stack and what each layer provides

Layer	Technology	What it provides
Front end	Next.js 16 App Router, React 19, Tailwind CSS 4	Marketing site, marketplace, booking flow, admin console, portals — one deployable
Writes	Next.js server actions with Zod validation	Type-safe form handling; price and availability recomputed server-side
Data	PostgreSQL 17 on Supabase, Prisma 6 client	42-table schema covering property, bookings, ledger, operations, people
Integrity	SQL exclusion constraints and triggers	No double bookings; append-only ledger; idempotent payments
Auth	NextAuth v5 — staff password + phone OTP	Role-based access from SUPER_ADMIN to VENDOR; suspended-user checks

Layer	Technology	What it provides
Hosting	Vercel (app) + GitHub Actions (keep-alive)	Automatic deploys from main; scheduled DB warm-up and cron sweeps
Money rail	Gateway abstraction with simulation mode	Same code path for simulated checkout and live Paystack webhook

4. What Has Been Built

The platform is organised into twelve modules. Each one is live in production today, and each was verified page-by-page and flow-by-flow during the 11 November health check. The screenshots in this section are taken from the live system.

4.1 Public Website

The marketing site presents the portfolio with a hero slideshow of current homes, amenity and location pages, a gallery, a mortgage calculator for prospective buyers, a social feed and contact channels, all bilingual-ready (English first, i18n scaffolded). Every page is server-rendered with per-page metadata, an automatic sitemap and robots configuration. The site works equally as a brochure and as the top of the booking funnel: the same units that staff manage in the console appear live on the site the moment they are published.

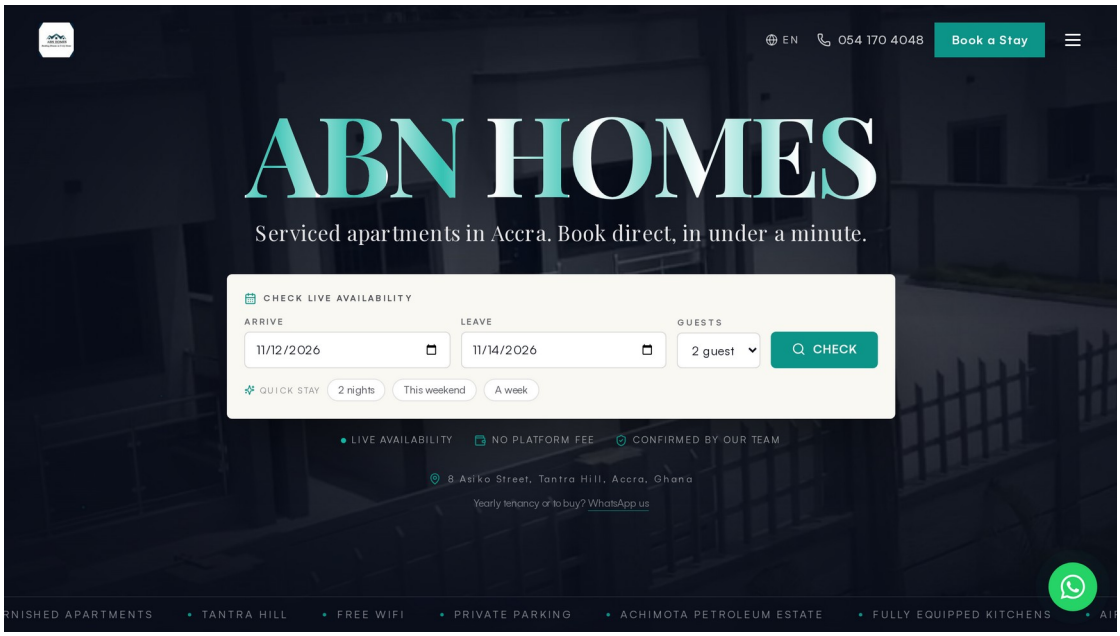


Figure 1: The public homepage with live homes from the platform inventory

4.2 Marketplace: Listings, Search and Reviews

The residences marketplace turns the internal unit inventory into a browsable storefront. Units flagged published appear with photos, features, sleeping arrangements, safety

disclosures and house rules; the stay detail page shows a live nightly quote from the unit's active rate plans, an availability preview calendar, published guest reviews with per-category sub-scores, a wishlist button for signed-in users, and similar-stay recommendations. Search supports date, guest, bedroom, amenity and price filtering with ranked sorting.

Listing URLs are slug-based, and the system is resilient about them: a unit's public URL is its stored slug if one exists, or a computed property-plus-code fallback if it does not — and the detail lookup accepts both shapes. This specific gap caused a real production bug on 11 November (every marketplace card linked to a 404 because stored slugs had never been written); it was fixed the same day both by backfilling the six existing units' slugs and by hardening the lookup so the failure cannot recur.

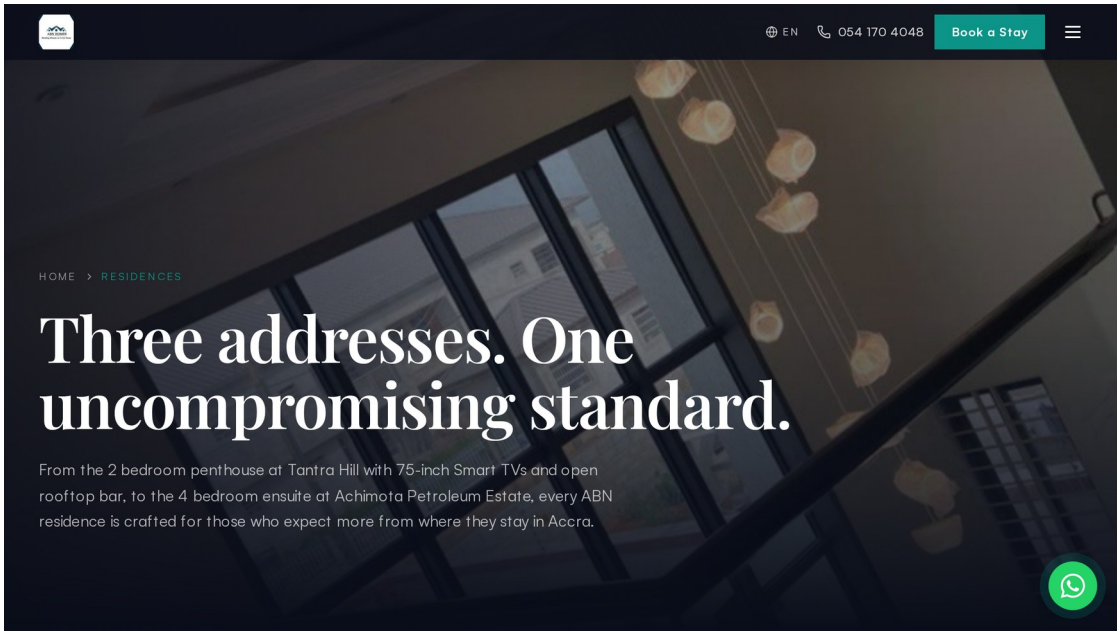


Figure 2: Marketplace grid with filters over live, published units

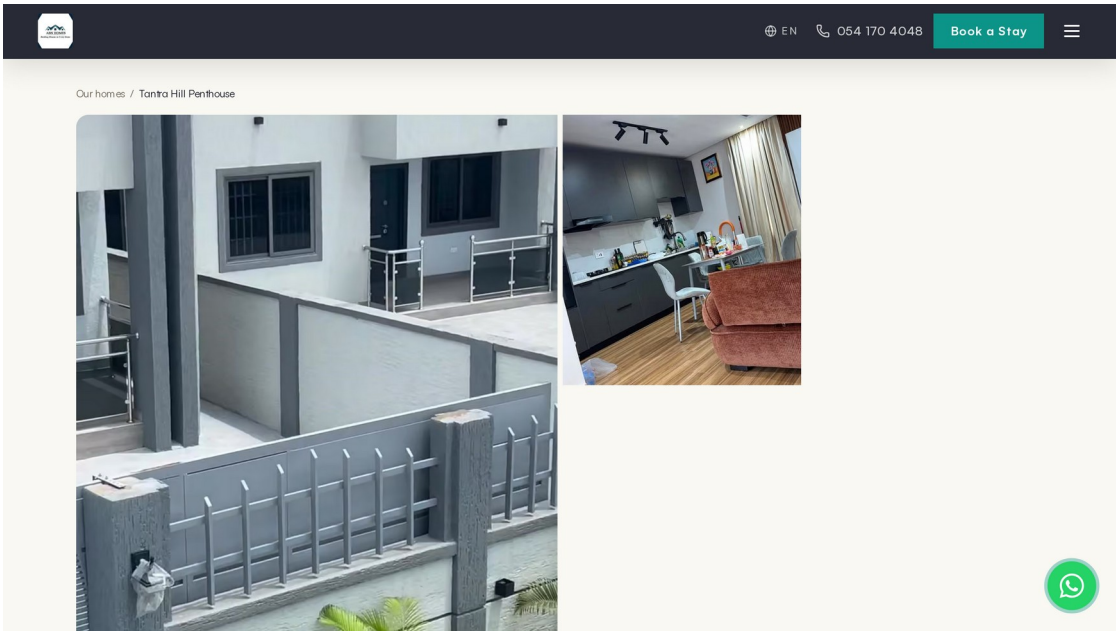


Figure 3: Stay detail page: gallery, quote from active rate plans, availability and reviews

4.3 Booking Engine

The booking engine is the most carefully guarded write path in the system because it is the only unauthenticated one. Nothing the browser sends is trusted: the price is recomputed from rate plans, availability is re-checked inside the booking transaction, and the unit must genuinely be published and bookable. A submitted request creates a PENDING hold that reserves the dates for 48 hours, gives the guest a booking code, and queues confirmation messages.

Overlapping holds are impossible by construction. The application checks availability twice — before and inside the transaction — and beneath that, a PostgreSQL exclusion constraint on unit and date range makes a double booking fail at the database even if two requests race in the same second. Add-ons and coupon codes are validated inside the same transaction, so a double submission can never double-apply a discount. Each confirmed booking also receives a portal access code that unlocks the guest's trip page without an account.

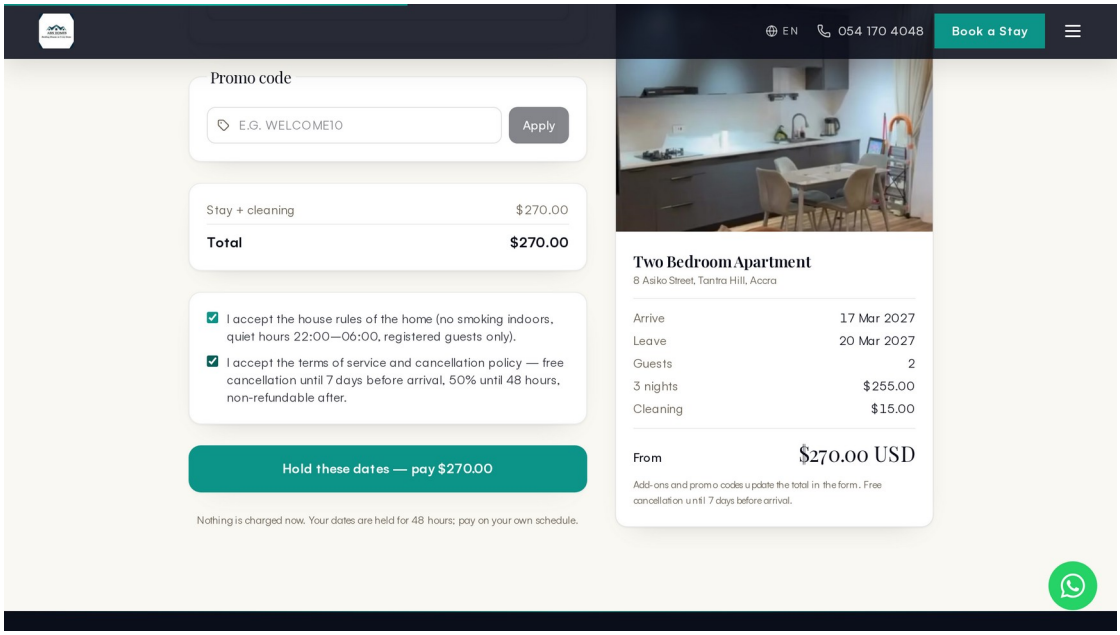


Figure 4: Booking request form: identity, arrival details and consents with all-inclusive quote

4.4 Payments

Payments sit behind a gateway abstraction with two implementations behind one interface: a live Paystack integration (mobile money and card channels, signature-verified webhook, reference-based reconciliation) and a simulation rail used while merchant keys are absent. The simulated checkout is not a mock-up — it writes a real payment, a real ledger entry and real allocations, then fires exactly the same receipt and auto-confirm logic the live gateway does. Switching to production is an environment-variable change, not a code change.

The pay page offers full or part payment across MTN Mobile Money, Telecel Cash, AirtelTigo Money, card and bank transfer channels, with a promo-code field. Settlement verifies the amount against the payment intent, posts the payment through the ledger, allocates it across the booking's charges in date order, and drives the reservation's status forward automatically. Refunds post reversing entries and mark the payment REVERSED; the webhook endpoint rejects unsigned or mismatched-signature calls with 401.

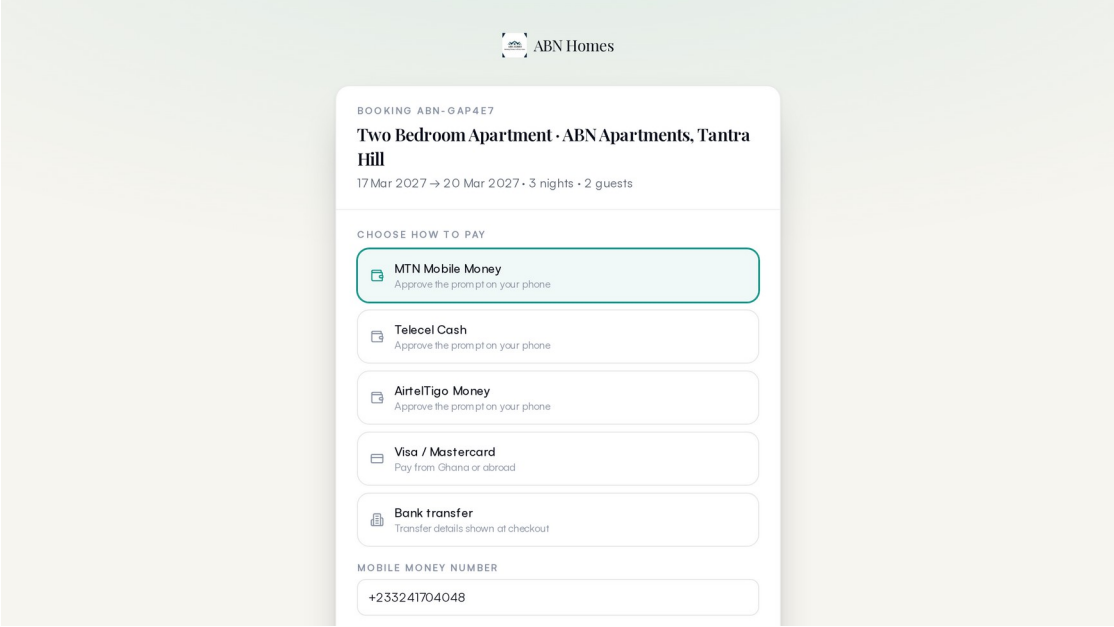


Figure 5: Checkout: channel choice, full or deposit payment, promo code

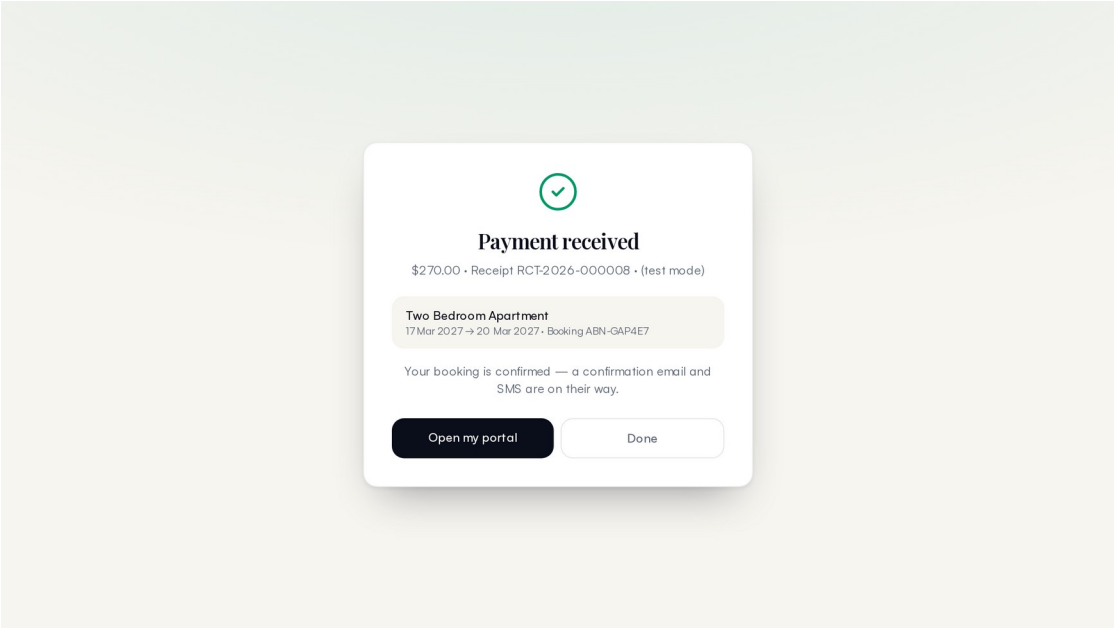


Figure 6: Settled payment: booking confirmed, receipt queued, ledger written

4.5 Money: the Double-Entry Ledger

Every financial fact in the platform flows through a double-entry ledger. A payment is not a status flag on a booking — it is a balanced pair of entries (a credit to the payment account, debits allocated to charges), linked to an immutable Payment record and an audit event. Charges, payments, refunds, expenses, payouts and rent invoices are all ledger kinds over the same primitive, which is what makes owner statements trustworthy: a statement is a query, not a reconstruction.

The database enforces what the application promises. Ledger entries are append-only at the trigger level — an attempt to update or delete one fails outright, and corrections are posted as reversing entries. During the 11 November work this guard fired exactly as designed when a raw test delete was attempted, and the correct path (cancel via the application, which posts reversals) was used instead. Around the ledger sit the practical money tools: expense capture with categories and vendor links, owner payout batches with approval states, and owner statements with balances.

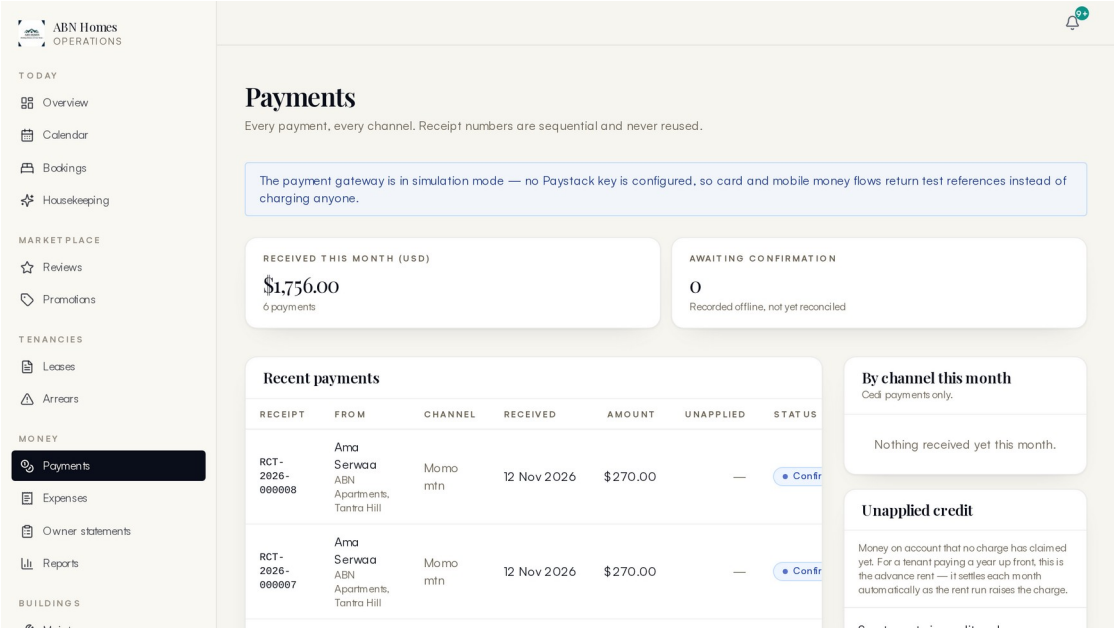


Figure 7: The money console: ledger, payments, expenses and payouts in one view

Table 2: Ledger primitives and where each is used

Primitive	Posted when	Guards
Charge	Booking confirmed, rent invoiced, fee applied	Reverse on cancellation; allocated by payments
Payment	Gateway settle, simulated settle, manual receipt	Idempotency key; verified amount against intent
Reversal	Refund or cancellation	References the entry it reverses; append-only pair
Expense	Operations spend captured with receipt	Category + vendor + property attribution
Payout	Owner withdrawal batch approved then paid	PENDING to APPROVED to PAID state machine

4.6 Operations: Housekeeping, Maintenance and Assets

Operations is where the platform earns its keep daily. The housekeeping board turns each check-out into a turnover task assigned to a named staff member with checklist items and completion states; the calendar view shows occupancy, arrivals and departures side by side so allocation decisions are made against real dates. Maintenance covers reactive tickets and preventive schedules (generators, AC servicing, pest control) with vendor assignment, while the asset registry ties equipment to units with service history. Every operations write lands in the audit trail.

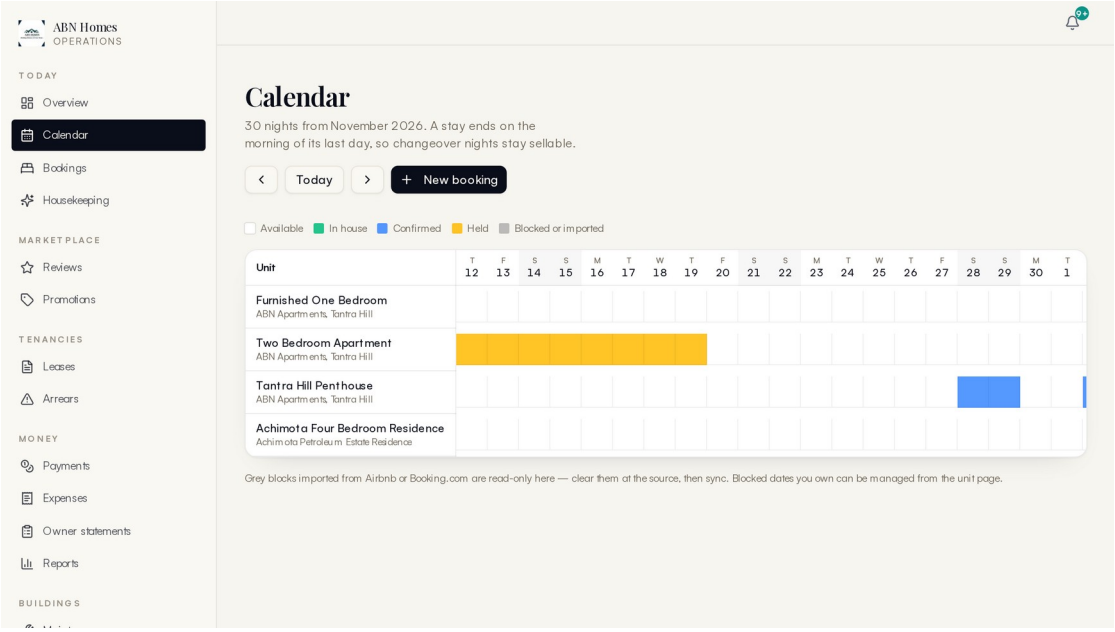


Figure 8: Occupancy calendar across units with arrivals and departures

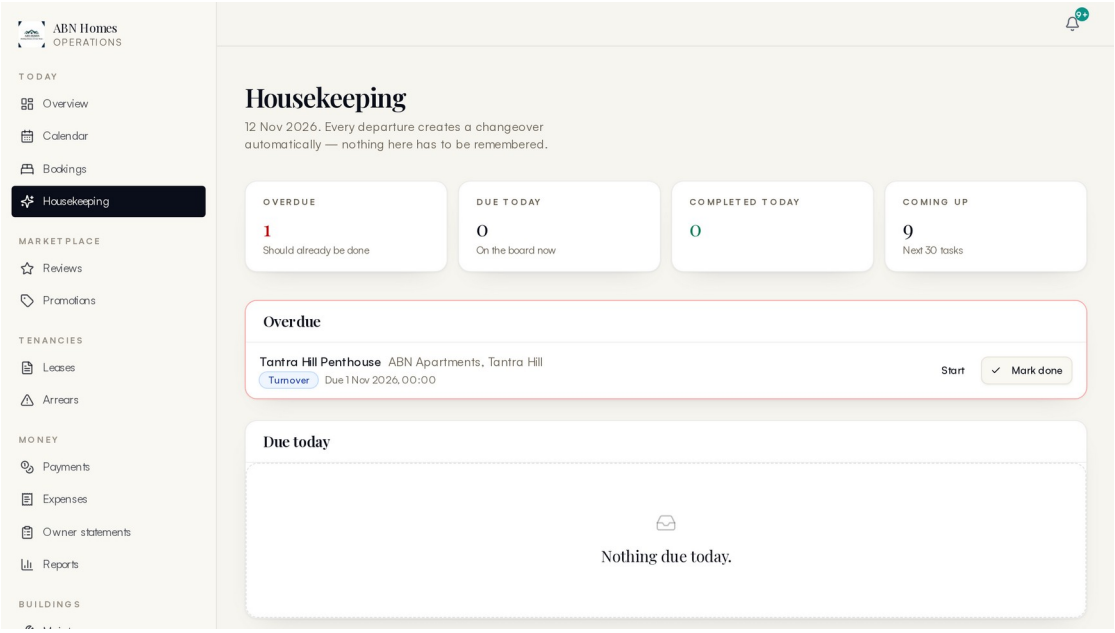


Figure 9: Housekeeping turnover board with task assignment and status

4.7 Guest Experience

After payment, a guest receives a portal access code that opens their trip page without needing an account. The trip hub gathers what a guest actually needs on the ground: arrival and departure details, access notes for the unit, Wi-Fi credentials, house rules, add-on purchases, and the booking's payment history. Arrival time and trip purpose captured at booking feed directly into front-desk preparation. Reviews are requested after check-out and moderated before publication — guests see only what staff have approved, and both guest and host sides of the review carry a role label.

4.8 Messaging and Notifications

All outbound communication flows through a database-backed outbox rather than being fired directly at providers. A booking event enqueues the right messages — email where an address exists, SMS either way, WhatsApp for channels that support it — each with a delivery status and attempt counter. A cron sweeper drains the outbox on schedule. While no SMS or email sender is configured, messages are marked sent in simulation mode with their full content recorded, so the communication design is testable end to end before any provider account exists.

In parallel, in-app notifications give staff a bell with per-user unread counts: booking requests, payment events, task assignments and system alerts arrive as bell items linked to the relevant record. Guests and owners see their own notification streams in their portals. The two paths — customer messages and staff notifications — are deliberately separate, because they have different lives: a message goes out once and is archived, a notification lives until read.

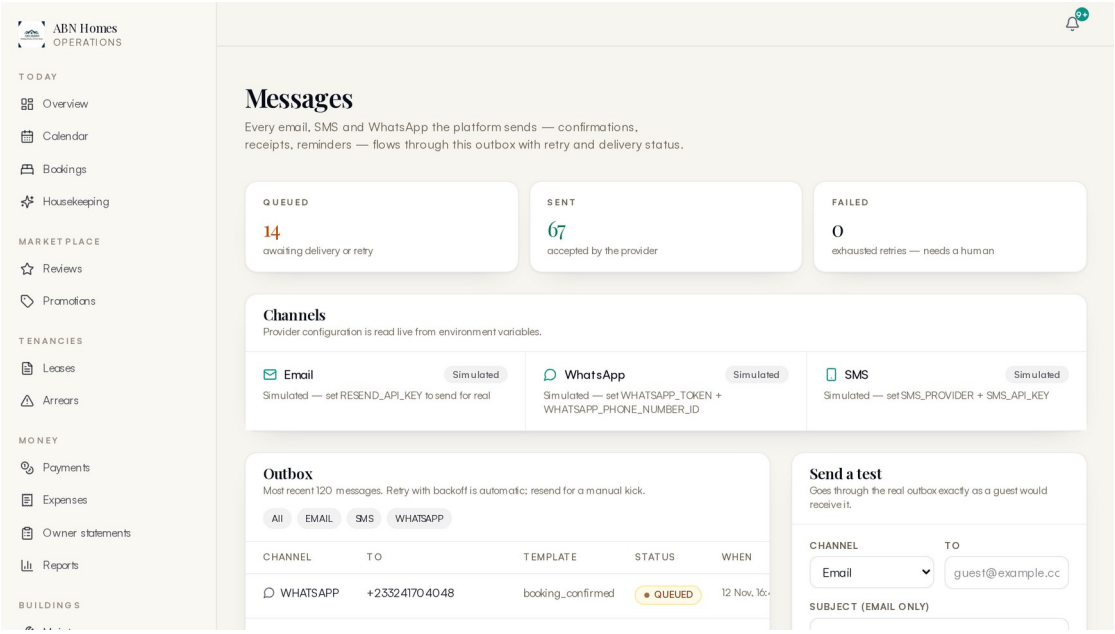


Figure 10: Message log with per-channel delivery status across the outbox

4.9 Admin Console

The admin console is the staff's single working surface: 22 pages spanning bookings (list, detail, creation), calendar, portfolio and per-unit pages, owners, leases, arrears, money, expenses, housekeeping, maintenance, assets, messages, reviews, promotions, reports, users, settings and audit. Pages are role-gated middleware-side — a housekeeper signing in lands with housekeeping powers only, a finance user sees money pages, and SUPER_ADMIN sees everything, including the user management and suspension tools.

Every admin page was swept during the 11 November verification with a signed-in SUPER_ADMIN session and returned clean: 22 of 22 pages, including the dynamic booking detail and unit portfolio routes. Reports and exports (occupancy, revenue, arrears) render server-side and export to CSV for finance.

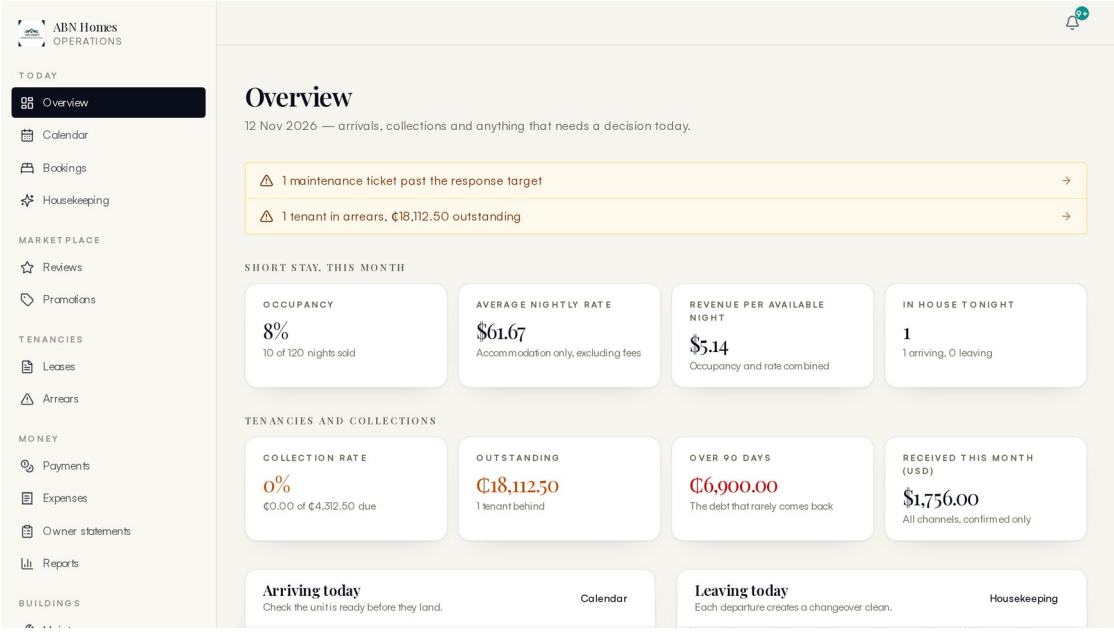


Figure 11: Admin landing: operational snapshot with alerts and quick actions

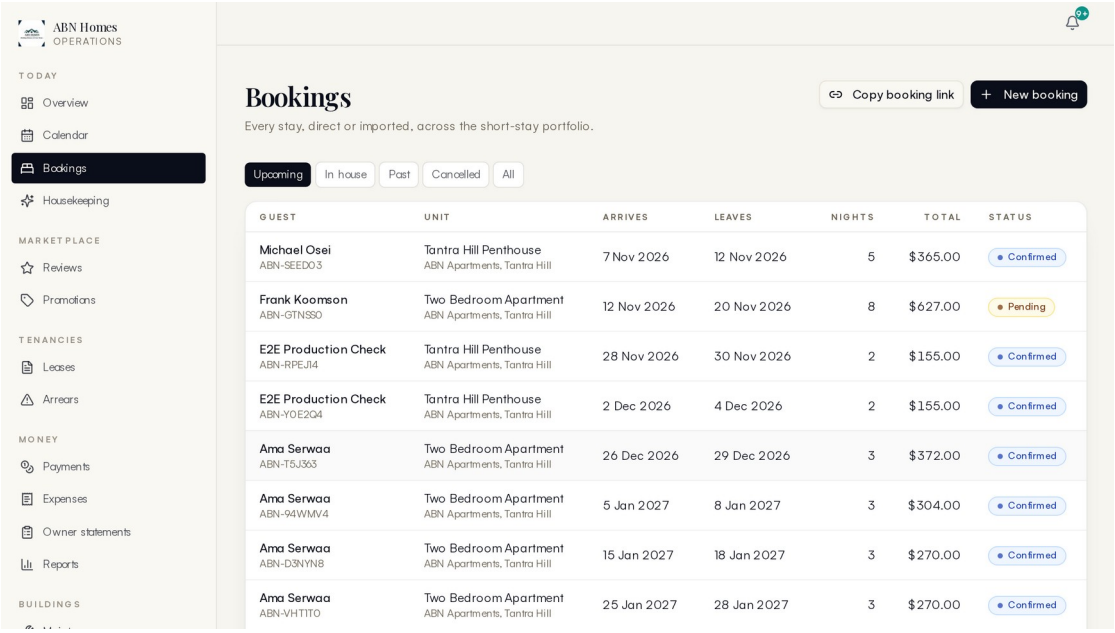


Figure 12: Bookings list with status, guest and balances

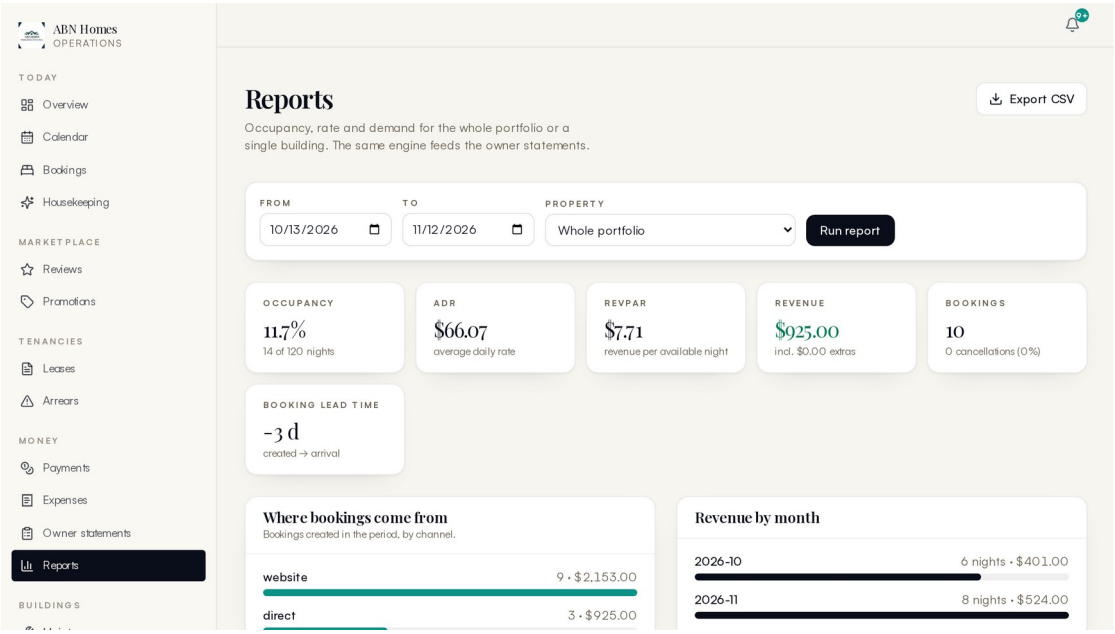


Figure 13: Reporting suite with CSV exports

4.10 Portals: Owner, Tenant and Vendor

Three role portals give stakeholders self-service without exposing the admin console. Owners see their properties' performance: occupancy, revenue, statements, expenses charged against their units and payout status, plus documents. Tenants (long-let residents) see their lease, rent account with arrears position, payment history and a request desk for maintenance issues, with messages back from the team. Vendors get the jobs assigned to them with acceptance and completion updates. All three sign in with the same phone OTP as guests — one identity system, role-scoped views.

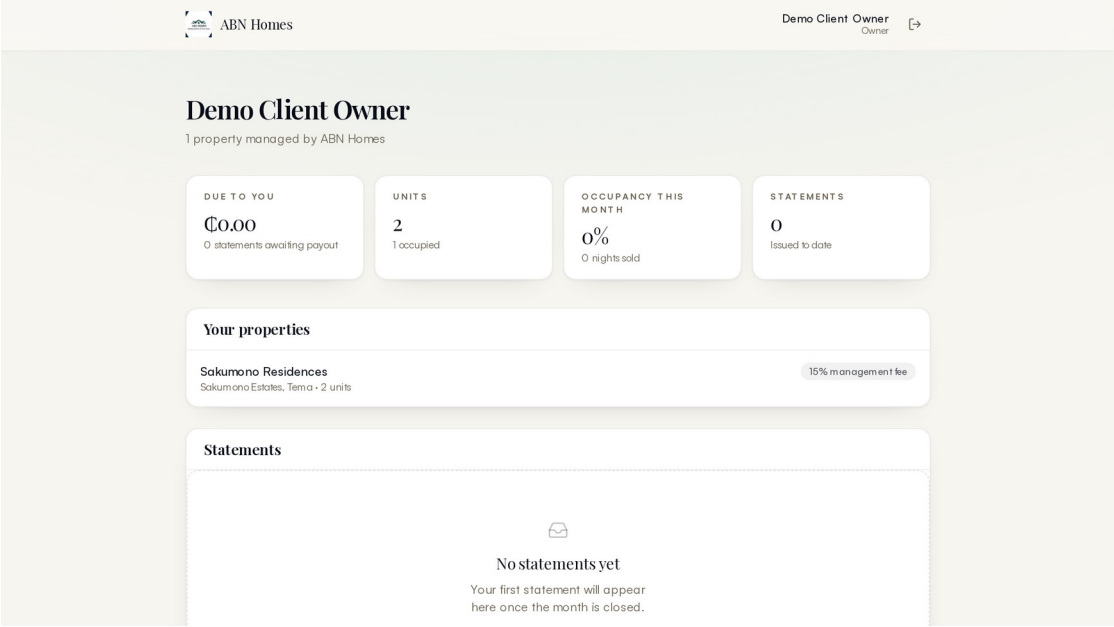


Figure 14: Owner portal: portfolio performance, statements and payouts

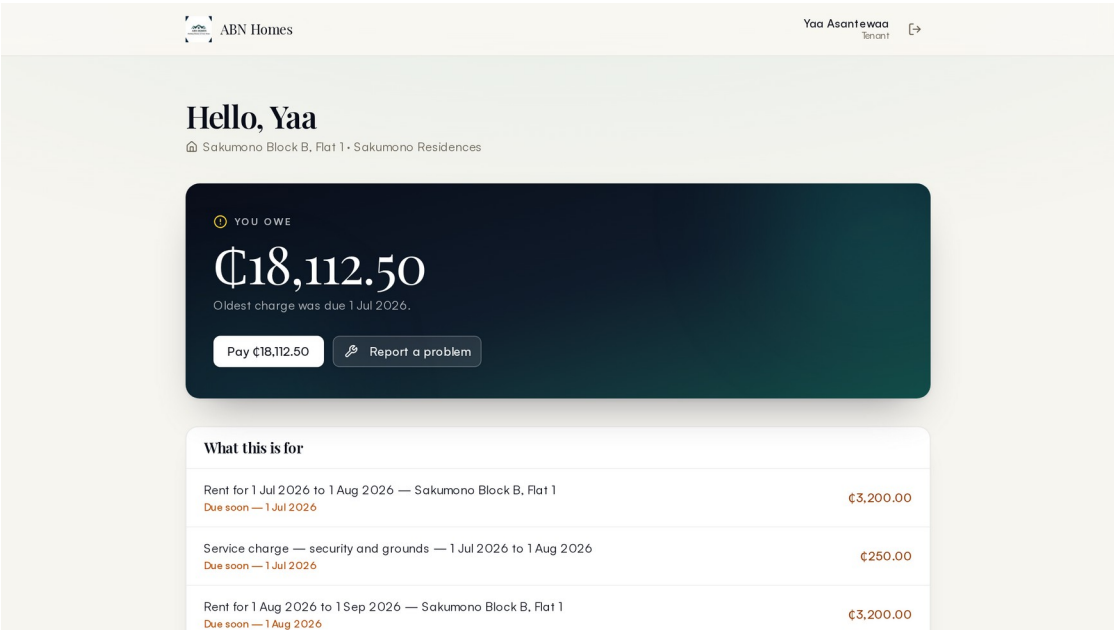


Figure 15: Tenant portal: lease, rent account and requests

4.11 Automation and Scheduled Jobs

Five scheduled jobs keep the system honest without human attention. The sweeper expires stale unpaid holds so dates return to sale automatically; the outbox drain pushes queued messages; the rent-run invoices every active long-let lease for the period and posts the charges; the iCal sync pulls external calendar feeds so OTA bookings block local dates; and the Supabase keep-alive workflow pings the database to prevent free-tier suspension. Cron routes authenticate with a bearer secret and reject everything else — verified with 401s during the health check.

Table 3: Scheduled jobs and their schedules

Job	Schedule	What it does
Sweeper	Vercel cron daily + GitHub 15-min	Expires 48h unpaid holds; drains the message outbox
Outbox drain	With sweeper	Sends queued SMS/email/WhatsApp with retry counter
Rent run	Daily 06:00 UTC	Invoices active long-let leases; posts rent charges
iCal sync	Daily 06:17 UTC	Pulls external OTA calendars; blocks booked dates
Keep-alive	GitHub Actions every 6h	Prevents Supabase free-tier pausing on idle

4.12 Security and Integrity

Security is layered rather than bolted on. Route protection is middleware-based with a role-to-path matrix; suspended users are locked out at the credential check, not the UI. The public booking path recomputes everything server-side and never trusts client prices. Webhooks verify signatures; cron routes require bearer secrets; OTP codes are hashed with expiry and attempt limits. Every staff write lands in an append-only audit event with actor, entity, before and after snapshots. The database itself is the last line: exclusion constraints prevent double bookings and append-only triggers prevent ledger tampering — both proven live during the November verification work.

5. How It Works: the Guest Journey End to End

The clearest way to explain the platform is to follow one booking through it. The steps below are exactly what the production end-to-end script exercises, and what the demonstration video shows on the live system.

1. Discover. A visitor browses the marketplace, filters by date and guests, and opens a stay page whose nightly quote comes live from the unit's rate plans.
2. Hold. They submit the booking form. The server recomputes the price, re-checks availability inside a transaction backed by an exclusion constraint, creates a PENDING hold for 48 hours, issues a booking code, and queues a confirmation SMS and email in the outbox.
3. Pay. On their private payment page the guest picks a channel and amount. Settlement writes the payment, posts the double-entry ledger pair, and allocates the money across the booking's charges oldest-first.

4. Confirm. The after-payment pipeline auto-confirms the booking, queues the receipt and the team alert, and issues the guest's portal access code — all without staff involvement.
5. Stay. The trip page shows access notes, Wi-Fi and house rules; front desk sees arrival time and purpose; housekeeping sees the future check-out task on its board.
6. Review. After check-out the guest is prompted to review their stay; published reviews appear on the marketplace page with sub-scores.
7. Report. The money is already inside owner statements and the owner's portal — no month-end reconstruction.

The money trail of the verified run: a 155.00 USD two-night quote produced one CREDIT ledger entry on the payment and two charge allocations, three outbox messages (email, SMS, WhatsApp), three staff bell notifications, and a full refund executed afterwards posted a reversal entry and returned the payment to REVERSED — every step visible in the admin console and audit log.

6. How It Works: Operations Journeys

Long-let renting runs on a monthly cycle: on schedule the rent-run invoices every active lease, posting a rent charge to the ledger; tenant payments arrive through the pay flow or are receipted manually; arrears surfaces automatically from the balance between charges and payments, with the arrears console listing tenants by amount owed; and owner statements aggregate the property's income and expenses for the period. A tenant's rent account is therefore always a live query, never a stale spreadsheet.

Turnover follows check-out: the housekeeping board receives the task automatically, staff complete checklists, and the unit flips back to AVAILABLE when the task closes. Maintenance blends reactive tickets (a tenant request can create one) with preventive schedules; costs captured as expenses flow into the property's statement. Owner payouts are prepared in batches, approved by a second pair of eyes, then marked paid — each state change audited.

7. Data Model Overview

The schema holds 42 tables in eight domains, all migrated to production and verified in sync with the Prisma schema on 11 November. Domain boundaries mirror the module boundaries, which keeps each area legible on its own.

Table 4: Schema domains and their principal tables

Domain	Principal tables	Notes
Portfolio	Property, Unit, RatePlan, AddOn, Amenity	Inventory the site sells from
People	Party, User, PropertyAccess, Referral	Parties are humans; Users are logins with roles
Stays	Reservation, ReservationAddOn, Review, WishlistItem	Short-stay bookings and social proof
Money	LedgerEntry, Charge, Payment, Allocation, PaymentIntent, IdempotencyRecord	Double-entry core with replay protection
Long lets	Lease, Tenant, RentInvoice, ArrearSnapshot	Annual renting with arrears tracking
Operations	HousekeepingTask, MaintenanceTicket, PreventiveMaintenance, Asset, Vendor (Party)	Daily facility running
Comms	MessageLog, Notification, OneTimeCode	Outbox plus staff bell plus OTP codes
Growth & admin	Coupon, CouponRedemption, Promotion, Expense, Payout, OwnerStatement, AuditEvent	Marketing and finance tools

8. Deployment and Operations

Deploys are push-to-main: GitHub receives the commit, Vercel builds and serves, typically inside three minutes. The production database is Supabase; schema changes ship with prisma db push followed by the constraint file (npm run db:constraints), and both steps are documented in the repository's deployment guide. Environment configuration is split between build-time public variables and server secrets; the documentation names each variable and where it belongs.

Two operating cautions stand. First, Prisma interactive transactions run with a 20-second timeout after the production incident described below — any new transaction doing multi-step money work should reuse the shared TX_OPTS constant rather than the default. Second, the Supabase database password and any tokens that have appeared in chat history are considered burned for a prototype and should be rotated before real customer data lands in the system.

9. Verification and Quality

Quality is enforced by three layers that run at different tempos. The unit test suite (37 tests) covers pricing, availability, ledger and coupon logic and runs on every change. The production end-to-end script (scripts/e2e_journey.ts) drives the real guest journey against the live database and asserts thirteen outcomes, from hold creation through refund reversal. And a manual sweep checks every page route with real sessions — all 33 routes passed on 11 November.

Table 5: Production end-to-end verification, 11 November 2026

Step	Assertion	Result
Hold	PENDING reservation created with live quote	PASS
Payment start	Simulation intent stored at exact amount	PASS
Settlement	Payment CONFIRMED; booking auto-confirmed	PASS
Ledger	1 credit entry + 2 charge allocations	PASS
Messages	Email, SMS, WhatsApp queued in outbox	PASS
Notifications	3 staff bell notifications created	PASS
Refund	Payment REVERSED; reversal entry posted	PASS

That same verification day caught two real production defects, both fixed and deployed. First, every marketplace card linked to a 404 because stored slugs were never written for existing units — fixed by backfilling slugs and accepting the computed shape in the lookup. Second, and more seriously, the public booking transaction exceeded Prisma's 5-second default timeout over the pooled connection to Supabase (measured 5.2 seconds), meaning real guest checkouts would fail intermittently; the fix raised interactive transactions to 20 seconds across all 19 call sites and is exactly the kind of defect only an end-to-end run against production infrastructure can catch.

10. What Is Yet to Be Added

10.1 Go-Live Switches (before real customers)

The platform is structurally complete for launch; four switches separate the prototype from commercial operation. None requires code changes — all are credentials or one-line configuration.

Table 6: Go-live switches in recommended order

Switch	What it changes	Effort
Paystack live keys	Turns the simulated checkout into real mobile-money and card settlement; webhook already signature-verified	Env vars
SMS sender account (Arkesel/Hubtel)	Delivers OTP codes and guest messages as real SMS instead of simulation	Env vars
Email sender (Resend/SES)	Delivers receipts and confirmations as real email	Env vars
Credential hygiene	Rotate the Supabase DB password, change the seeded staff password (public in the repo), sync CRON_SECRET between Vercel and GitHub, revoke tokens shared in chat	1 hour

10.2 Product Roadmap

Beyond the go-live switches, the roadmap below is sequenced by business value. Items near the top remove friction from money and communication; later items grow distribution and insight.

Table 7: Roadmap by priority

Area	Feature	Why it matters
Payments	Paystack subaccount per property; payout reconciliation view	Clean per-owner money separation at scale
Comms	WhatsApp Business API for trip updates	Ghana guests live on WhatsApp; richest channel
Distribution	Outbound iCal to Airbnb/Booking.com	Fill off-peak nights from OTAs while keeping direct bookings
Growth	Referral rewards activation; promo campaign tooling	Referral table exists; needs the loop switched on
Ops	Photo uploads on housekeeping/maintenance tasks	Evidence trail for turnover quality and repairs
Insight	Occupancy and revenue trend dashboard; export scheduling	Owners and management see trajectory, not just balances
Platform	Image pipeline (resize/WebP); PWA push notifications	Faster media pages; nudge without email

10.3 Known Prototype Caveats

Four things are intentionally accepted in the prototype state. OTP sign-in codes are returned in the API response while no SMS gateway exists, so phone-based sign-in must be treated as open until the sender account is configured. Seeded staff accounts share a repo-public password. The demo bookings created during verification (guest Ama Serwaa) remain in the database as evidence and can be cancelled from the admin console. And the 15-minute GitHub sweeper authenticates with a CRON_SECRET that does not yet match Vercel's value — the daily Vercel cron covers the same work in the interim, so nothing is actually stuck.

Appendix A. Route Inventory (verified 11 November 2026)

Table 8: Public site routes

Route	Purpose
/, /about, /amenities, /gallery, /locations, /contact	Marketing pages
/residences, /stays/[slug]	Marketplace grid and stay detail
/book, /book/confirm	Availability search and booking request
/pay/[code], /payments/simulate, /payments/callback	Checkout and settlement
/signin	Staff, phone-OTP and booking-code sign-in
/portal/owner, /portal/tenant, /portal/vendor	Role portals
/sitemap.xml, /robots.txt, /manifest	SEO and PWA metadata

Table 9: Admin console pages (22, all verified 200 with SUPER_ADMIN session)

Group	Pages
Bookings	/admin/bookings, /bookings/new, /bookings/[id]
Portfolio & owners	/admin/portfolio, /portfolio/[unitId], /admin/owners, /admin/leases, /leases/new, /admin/arrears
Money	/admin/money, /admin/expenses, /admin/reports
Operations	/admin/calendar, /admin/housekeeping, /admin/maintenance, /admin/assets
Comms & growth	/admin/messages, /admin/reviews, /admin/promotions
Governance	/admin/users, /admin/settings, /admin/audit

API surface: /api/auth (NextAuth + OTP), /api/webhooks/paystack (signature-verified), /api/cron/outbox, /api/cron/rent-run, /api/cron/ical-sync (bearer-secret guarded), /api/notifications and /api/reports/export (session-guarded), /api/calendar/[unitId] (public iCal feed), /api/enquiries, /api/subscribers and /api/tours (public lead capture).

Repository tooling worth knowing about: scripts/e2e_journey.ts (production guest-journey verification), scripts/backfill_unit_slugs.ts (slug backfill), scripts/cancel_demo_hold.ts (demo booking cleanup through the proper cancel path), scripts/db_query.ts (read-only ad-hoc queries), and prisma/sql/001_constraints.sql (the database-level integrity layer).